

INSTITUTO TECNOLÓGICO DE CULIACAN

INGENIERIA EN SISTEMAS COMPUTACIONALES

Construcción de un modelo de Reconocimiento Facial

TOPICOS DE IA

DENZEL ARTURO GUTIERREZ CHAVEZ

JUAN CARLOS QUIÑONEZ MADRID



TECNOLÓGICO
NACIONAL DE MÉXICO

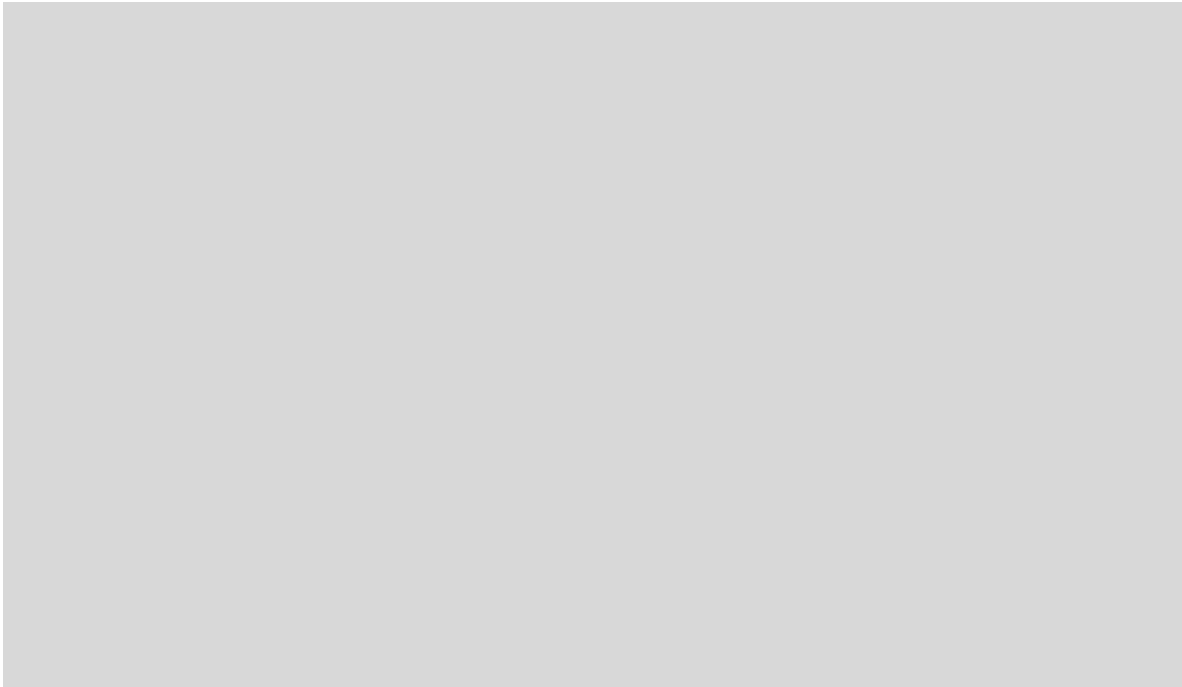


CULIACAN, SINALOA

30/05/26

Link repo: <https://github.com/CarlosMadrid11/Topicos-de-la-IA>

Link video de pruebas: <https://youtu.be/pchc1KMdRQQ>



INSTRUCCIONES

Para el desarrollo de este proyecto sobre el Desarrollo de un modelo de Reconocimiento Facial proporcionado por nuestro profesor, planteamos las siguientes instrucciones a construir:

Instala y configura el entorno de desarrollo necesario para la construcción de la aplicación móvil. Verifica la correcta instalación de Flutter SDK, Android Studio, el emulador Android y los complementos requeridos para el desarrollo de aplicaciones Flutter. Comprueba que el entorno permita la compilación y ejecución de proyectos sin errores.

Crea un nuevo proyecto Flutter denominado **reconocimiento_facial** y configura la estructura inicial de la aplicación. Integra las dependencias necesarias para la ejecución de modelos TensorFlow Lite, la captura o selección de imágenes desde el dispositivo móvil y el procesamiento previo de imágenes para su análisis mediante inteligencia artificial.

Integra dentro del proyecto los recursos generados en fases anteriores, incluyendo el modelo de reconocimiento facial entrenado, el archivo de mapeo de clases y el conjunto de datos de alumnos. Configura adecuadamente el archivo de recursos para asegurar que todos los elementos puedan ser cargados y utilizados durante la ejecución de la aplicación.

Diseña e implementa una pantalla de reconocimiento facial que permita al usuario seleccionar una imagen desde la galería o capturar una fotografía utilizando la cámara del dispositivo. Desarrolla el proceso de preprocesamiento de imágenes, considerando el redimensionamiento a 224×224 píxeles y la normalización de los valores de entrada. Ejecuta el modelo TensorFlow Lite utilizando la imagen procesada y obtén la predicción correspondiente al alumno identificado.

Implementa los mecanismos necesarios para interpretar la salida del modelo de inteligencia artificial, identificando la clase con mayor probabilidad y asociándola con la información almacenada en los archivos de configuración. Muestra en pantalla el nombre del alumno reconocido y el porcentaje de confianza obtenido durante la predicción.

Desarrolla una segunda pantalla destinada a la visualización de la información detallada del alumno reconocido. Recupera y presenta los datos correspondientes al número de control, nombre completo, carrera y edad, asegurando una navegación adecuada entre ambas interfaces de usuario.

Realiza pruebas funcionales utilizando un emulador Android configurado en Android Studio. Verifica el correcto funcionamiento de la selección de imágenes, el procesamiento de datos, la ejecución del modelo de reconocimiento facial y la visualización de la información de los alumnos. Documenta los resultados obtenidos y registra una demostración del funcionamiento de la aplicación para evidenciar el cumplimiento de los objetivos planteados.

Finalmente, documenta cada etapa del desarrollo realizada durante la construcción de la aplicación, justificando las decisiones tomadas en la integración del modelo de inteligencia artificial, el procesamiento de imágenes y el diseño de la interfaz. Incluye evidencias de funcionamiento, resultados obtenidos y una descripción detallada de la arquitectura implementada en la solución móvil.

Empezamos mencionando que este proyecto consta de distintas fases en las cuales se involucran para poder llevar a cabo este proyecto. Comenzamos en la Fase 0 donde analizamos lo siguiente:

Realizamos una verificación del entorno de desarrollo para confirmar la correcta instalación de TensorFlow y conocer la versión disponible para el proyecto. Además, comprobamos la disponibilidad de una GPU compatible, ya que su utilización permite acelerar significativamente los procesos de entrenamiento y ejecución de modelos de inteligencia artificial.

```
import tensorflow as tf

print('TensorFlow version:', tf.__version__)

gpus = tf.config.list_physical_devices('GPU')
if gpus:
    print(f' GPU ya estaa disponible: {gpus[0].name}')
else:
    print('No se detectó GPU Ve a Entorno de ejecución → Cambiar tipo → T4 GPU')

TensorFlow version: 2.20.0
```

Realizamos la importación de las librerías necesarias para el desarrollo del proyecto y verificamos las versiones instaladas de cada una. Esta comprobación permitió asegurar la compatibilidad entre las herramientas utilizadas para el procesamiento de imágenes, el manejo de datos, la visualización de información y la construcción del modelo de reconocimiento facial.

```
Versiones de librerías:
tensorflow: 2.20.0
numpy: 2.0.2
opencv: 4.13.0
scikit-learn: 1.6.1
seaborn: 0.13.2
matplotlib: 3.10.0
Pillow: 11.3.0
pandas: 2.2.2
```

Construimos la estructura de directorios para organizar los recursos del proyecto de manera ordenada. Esto permitió separar los conjuntos de datos, modelos entrenados, métricas y archivos destinados a la aplicación móvil, facilitando la gestión y el mantenimiento de cada etapa del desarrollo.

```
reconocimiento_facial_tec_culiacan/dataset/raw
reconocimiento_facial_tec_culiacan/dataset/augmented
reconocimiento_facial_tec_culiacan/dataset/processed/train
reconocimiento_facial_tec_culiacan/dataset/processed/val
reconocimiento_facial_tec_culiacan/dataset/processed/test
reconocimiento_facial_tec_culiacan/modelo
reconocimiento_facial_tec_culiacan/metricas
reconocimiento_facial_tec_culiacan/app_data
```

Con esto mas que claro comenzamos y nos anunciamos a la Fase 1, donde ya empezaremos a trabajar con estas dependencias.

FASE 1

Construimos una base de datos inicial con la información de los 20 alumnos que participarían en el sistema de reconocimiento facial. Además, asociamos las rutas de las imágenes correspondientes a cada estudiante y generamos un archivo CSV que serviría como fuente de consulta para relacionar las predicciones del modelo con los datos personales de cada alumno.

no_control	nombre	carrera
22170852	Marcelo Vidrio Ochoa	Ingeniería en Sistemas
21170437	Juan Carlos Quiñonez Madrid	Ingeniería en Sistemas
22170679	Denzel Arturo Gutierrez Chavez	Ingeniería en Sistemas
22170642	Edwin Valentin Dominguez Lizarraga	Ingeniería en Sistemas
22170560	Carlos Eduardo Angulo Inzunza	Ingeniería en Sistemas
22170759	Jose Carlos Ozuna Tirado	Ingeniería en Sistemas
22170608	Edgar Abraham Cervantes Alfaro	Ingeniería en Sistemas
21170420	Carlos Leonardo Padilla Gonzalez	Ingeniería en Sistemas
21170445	Marian Gabriela Ramirez Pompa	Ingeniería en Sistemas
22170818	Luis Daniel Sanchez Ramirez	Ingeniería en Sistemas
22170828	Jose Manuel Sotelo Cuevas	Ingeniería en Sistemas
22170727	Bryan Najera Beltran	Ingeniería en Sistemas
22170657	Jeylen Isai Garcia Feliz	Ingeniería en Sistemas
21170260	Edgar Armando Beltran Ochoa	Ingeniería en Sistemas
20170439	Giovanni Gonzalez Pulgarin	Mecatrónica
21170363	Jorge Alberto Lara Beltran	Ingeniería en Sistemas
22170736	Javier Uriel Murillo Araiza	Ingeniería en Sistemas
22170136	Emilio Arturo Ruiz Baez	Ingeniería Eléctrica
25170001	Yahir Ortega Piña	Ingeniería Industrial
22170708	Juan Emilio Madrid Alvarez	Ingeniería en Sistemas

Despues de eso, agregamos las fotos en el Drive donde las mantenemos y mandamos al modelo a analizarlas y revisar que estén listas para el Data Argumentation.

Implementamos diversas funciones de transformación de imágenes con el objetivo de aumentar artificialmente la cantidad y variedad de datos disponibles para el entrenamiento. Mediante la combinación aleatoria de estas transformaciones, logramos simular distintas condiciones de captura, contribuyendo a que el modelo aprendiera características faciales más robustas y menos sensibles a cambios en el entorno.

```
Funciones de transformación definidas
Transformaciones disponibles:
• Rotación ±15°
• Brillo ±30%
• Zoom 85-115%
• Desplazamiento
• Ruido gaussiano
• Escala de grises
• Flip horizontal
```

Realizamos el proceso de aumento de datos generando 50 variantes artificiales por cada fotografía original de los alumnos. Esta estrategia permitió ampliar significativamente el conjunto de entrenamiento, pasando de unas cuantas imágenes por estudiante a miles de ejemplos, lo que favorece un mejor aprendizaje y capacidad de reconocimiento por parte del modelo.

```
Marcelo_Vidrio      200 imágenes generadas
Juan_Quíñonez      200 imágenes generadas
Denzel_Gutierrez    200 imágenes generadas
Edwin_Dominguez     200 imágenes generadas
Carlos_Angulo       200 imágenes generadas
Jose_Ozuna          200 imágenes generadas
Edgar_Cervantes     200 imágenes generadas
Carlos_Padilla      200 imágenes generadas
Marian_Ramirez      200 imágenes generadas
Luis_Sanchez        200 imágenes generadas
Jose_Sotelo         200 imágenes generadas
Bryan_Najera        200 imágenes generadas
Deylen_Garcia       200 imágenes generadas
Edgar_Beltran       200 imágenes generadas
Giovanni_Gonzalez   200 imágenes generadas
Jorge_Lara          200 imágenes generadas
Javier_Murillo      200 imágenes generadas
Emilio_Ruiz         200 imágenes generadas
Yahir_Ortega        200 imágenes generadas
Juan_Madrid         200 imágenes generadas
```

Dando en total alrededor de 4000 imágenes listas para ser analizadas.

FASE 2 ---

Realizamos la división del conjunto de imágenes en tres subconjuntos: entrenamiento, validación y prueba, utilizando una distribución del 70 %, 15 % y 15 %, respectivamente. Esta separación permitió entrenar el modelo, ajustar su desempeño y evaluar su capacidad de generalización utilizando datos que no fueron vistos durante el proceso de aprendizaje.

Configuramos los generadores de datos encargados de cargar y preparar automáticamente las imágenes para el entrenamiento del modelo. Durante este

proceso, las imágenes fueron redimensionadas y normalizadas, además de organizarse por clases, permitiendo que la red neuronal pudiera identificar correctamente a cada alumno durante las etapas de entrenamiento, validación y prueba.

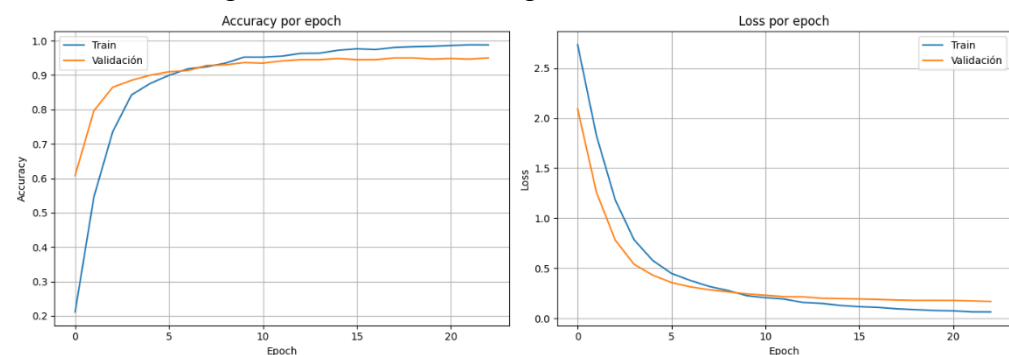
Construimos el modelo de reconocimiento facial utilizando la arquitectura MobileNetV2 mediante la técnica de *transfer learning*. Para aprovechar el conocimiento previamente adquirido en ImageNet, congelamos las capas base y añadimos nuevas capas densas especializadas en la clasificación de los 20 alumnos, reduciendo el tiempo de entrenamiento y mejorando la eficiencia del modelo.

```
Modelo construido
Parámetros totales      : 2,621,396
Parámetros entrenables  : 363,412 ← solo estos se actualizan
Parámetros congelados   : 2,257,984 ← conocimiento de ImageNet
```

Realizamos el entrenamiento del modelo utilizando los datos preparados previamente, configurando mecanismos de control para optimizar el proceso. Implementamos técnicas como *Early Stopping* y *Model Checkpoint* con el fin de evitar el sobreajuste y conservar automáticamente la versión del modelo que obtuvo el mejor desempeño durante la validación. Aquí obtuvimos un 0.95 de accuracy, siendo una noticia positiva.

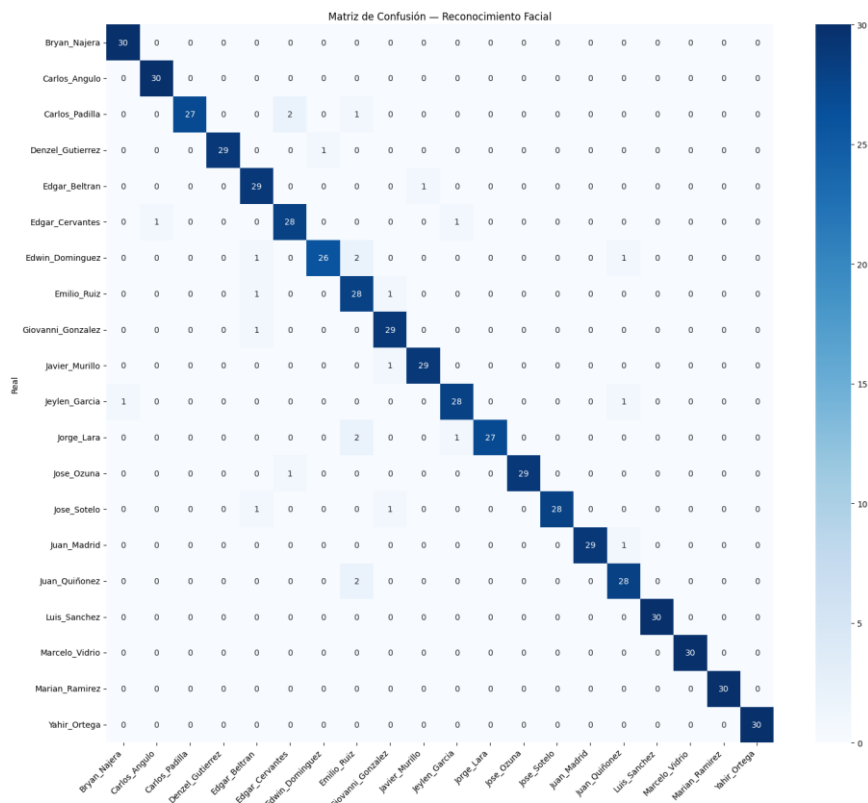
Evaluamos el desempeño del modelo utilizando el conjunto de datos de prueba para medir su capacidad de reconocimiento sobre imágenes no vistas previamente. Además, generamos gráficas de precisión y pérdida a lo largo de las épocas de entrenamiento, lo que permitió analizar visualmente el aprendizaje del modelo y detectar posibles problemas de sobreajuste o subajuste.

Dandonos el siguiente resultado en grafica:



de confusión y
as permitieron
como detectar

posibles errores de clasificación y analizar el rendimiento individual de cada clase.



Reporte por alumno:				
	precision	recall	f1-score	support
Bryan_Najera	0.97	1.00	0.98	30
Carlos_Angulo	0.97	1.00	0.98	30
Carlos_Padilla	1.00	0.90	0.95	30
Denzel_Gutierrez	1.00	0.97	0.98	30
Edgar_Beltran	0.88	0.97	0.92	30
Edgar_Cervantes	0.90	0.93	0.92	30
Edwin_Dominguez	0.96	0.87	0.91	30
Emilio_Ruiz	0.80	0.93	0.86	30
Giovanni_Gonzalez	0.91	0.97	0.94	30
Javier_Murillo	0.97	0.97	0.97	30
Jeylen_Garcia	0.93	0.93	0.93	30
Jorge_Lara	1.00	0.90	0.95	30
Jose_Ozuna	1.00	0.97	0.98	30
Jose_Sotelo	1.00	0.93	0.97	30
Juan_Madrid	1.00	0.97	0.98	30
Juan_Quinonez	0.90	0.93	0.92	30
Luis_Sanchez	1.00	1.00	1.00	30
Marcelo_Vidrio	1.00	1.00	1.00	30
Marian_Ramirez	1.00	1.00	1.00	30
Yahir_Ortega	1.00	1.00	1.00	30
accuracy			0.96	600
macro avg	0.96	0.96	0.96	600
weighted avg	0.96	0.96	0.96	600

Conclusion:

El principal obstáculo de esta fase fue la incompatibilidad entre Flutter 3.44, Gradle AGP 9.0 y los paquetes de inferencia disponibles (tflite_flutter, onnxruntime). A pesar de múltiples intentos ajustando versiones de JVM, AGP y regenerando el proyecto Android, el entorno de compilación nunca logró estabilizarse, en gran parte agravado por el uso de macOS 26 Tahoe (beta), que introdujo conflictos adicionales con libexpat que impedían incluso instalar Python correctamente fuera de conda.

Ante esto, se tomó la decisión de pivotar hacia una demo funcional en web usando Streamlit, lo que permitió utilizar directamente el modelo .keras entrenado en Colab sin ninguna conversión adicional, eliminando por completo la capa de Gradle y las dependencias nativas que bloqueaban el avance. El resultado es una aplicación funcional que demuestra el reconocimiento facial con el mismo modelo, la misma lógica de preprocesamiento y los mismos datos, cumpliendo el objetivo del proyecto aunque en una plataforma distinta a la originalmente planeada.